

Project 16 – The National Identity Verification Engine

Level: Advanced Beginner | Version 2.0 | Difficulty: 7/10

Unlocking Secure Authentication: A Closer Look at the Number Verify API

(Part 2 of my series – Sharing my perspectives on telco APIs & NaaS)

linkedin.com

1. Objective

Build a C program that accepts a fictional 12-digit identity number and determines whether it passes a predefined set of verification rules.

The program must identify every failed condition and explain why the number was rejected.

// Note: This is a fictional identification system. It has no connection to Aadhaar or any real government identity system.

2. Input Requirements

The program must accept exactly one identity number.

Property	Requirement
Length	Exactly 12 characters
Allowed characters	Digits 0–9 only
First digit	Cannot be 0
Spaces	Not allowed
Alphabets	Not allowed
Special characters	Not allowed
Negative sign	Not allowed

Examples:

Input	Result
482731905624	Passes input-format checks
012731905624	Reject

48273190562	Reject
4827319056247	Reject
48273190A624	Reject
4827 1905624	Reject
4827-1905624	Reject

! **Important:** Your program must distinguish an invalid character from an incorrect length. If multiple errors exist, report all applicable errors.

3. Mathematical Verification Conditions

Your fictional identity system has the following rules.

Condition A – Digit sum

The sum of all 12 digits must be divisible by 3.

> *Example:*
> *482731905624*
> *Digit sum = 4 + 8 + 2 + 7 + 3 + 1 + 9 + 0 + 5 + 6 + 2 + 4 = 51.*
> *51 is divisible by 3, so this condition passes.*

Condition B – Even and odd position sum

Positions are counted from left to right, starting at position 1.

- Add digits at odd positions: 1, 3, 5, 7, 9, 11.
- Add digits at even positions: 2, 4, 6, 8, 10, 12.

The absolute difference between these two sums must be divisible by 5.

> *For the example:*
> *- Odd-position sum = 4 + 2 + 3 + 9 + 5 + 2 = 25.*
> *- Even-position sum = 8 + 7 + 1 + 0 + 6 + 4 = 26.*
> *- Absolute difference = 1.*
> *1 is not divisible by 5, so this condition fails.*

Condition C – Repeated digit restriction

No digit may appear more than three times.

Examples:

- 111234567890 → Reject (digit 1 appears three times? Actually it appears three times, so this condition passes).

- 111123456789 → Reject (digit 1 appears four times).
- 123456789012 → Passes this condition.

Condition D – First and last digit relationship

The sum of the first and last digits must be greater than 5 and less than 15.

> *For 482731905624:*
> *4 + 4 = 8.*
> *This condition passes.*

---

4. Rejection Reasons

Your program must support these rejection messages:

Error code	Rejection reason
E01	Identity number contains an invalid character.
E02	Identity number must contain exactly 12 characters.
E03	Identity number cannot start with zero.
E04	Digit sum is not divisible by 3.
E05	Odd/even position sum difference is not divisible by 5.
E06	A digit appears more than three times.
E07	First and last digit sum is outside the permitted range.

If three conditions fail, the program must report all three errors rather than stopping at the first one.

5. Output Requirements

Your program should produce a report containing:

- Verification status: APPROVED or REJECTED.
- Input-format validation results.
- Mathematical verification results.
- Every applicable error code and its explanation.
- Total number of failed conditions.

Never display the complete identity number in the output. You may mask it, for example: 48XXXXXXXX24.

6. Testing Requirements

Your program must handle these test cases.

Test	Input	Expected result
T01	482731905624	Reject: E05
T02	123456789012	Reject: E04
T03	111123456789	Reject: E06
T04	012345678912	Reject: E03
T05	12345678901	Reject: E02
T06	12345678901A	Reject: E01
T07	111234567890	E06 must NOT be reported
T08	482731905624	E04 must NOT be reported

These are individual test expectations, not necessarily the complete set of errors for every input.

7. Programming Restrictions

For this project:

- Use C.
- You may use stdio.h, stdlib.h, unistd.h, ctype.h and string.h.
- Do not use loops.
- Do not use arrays.
- Do not create user-defined functions.
- Do not use structures.
- Do not use dynamic memory allocation.

You can use the conditional statements and pointer knowledge you've already demonstrated.

One final clarification: This is a rule-based fictional verification engine, not a real identity authentication system. Passing its mathematical rules only means that the input satisfies our invented specification.

Now the challenge is properly defined. You should be able to implement it without having to invent missing requirements yourself.